DAY 6 OF 7

Background jobs with Celery & Redis

Async CSV export, a daily reminder and a monthly activity report – without slowing down the API

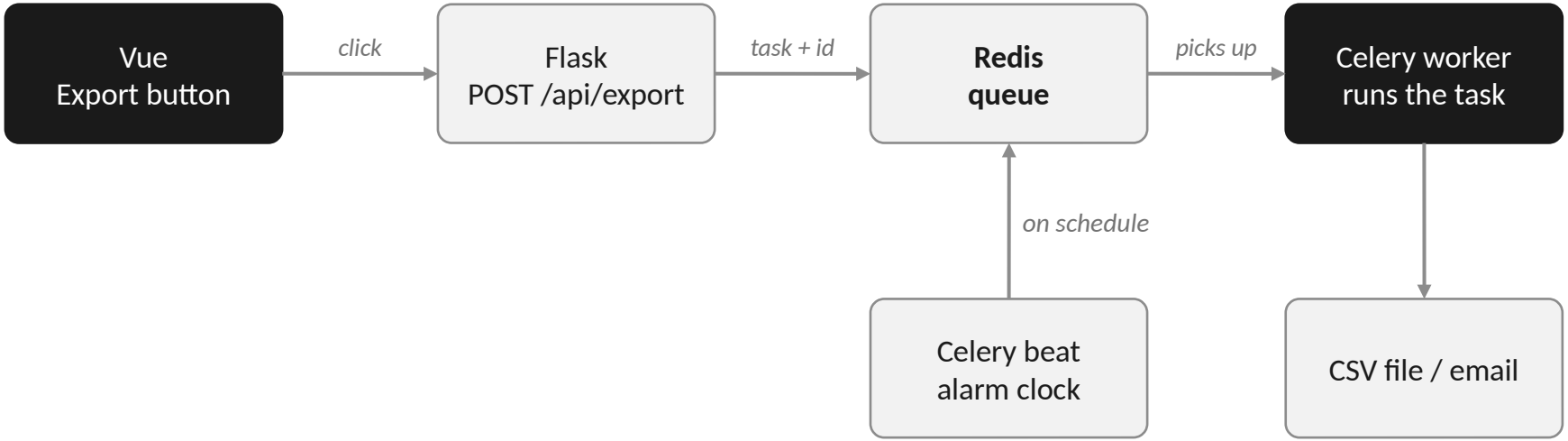
Today's plan

15 min	Why background jobs?
10 min	Start Redis (and MailHog)
20 min	Celery + Flask, start the worker
20 min	Job 1: export CSV + download button
25 min	Jobs 2 & 3: reminder and monthly report
30 min	Celery beat, run-now buttons, demo

By the end you can

- Explain why slow work must not run inside a request
- Run a Celery worker and Celery beat with Redis
- Write tasks that use the database and templates
- Start a job from Vue and download the result

How a background job runs



Flask answers at once with a task_id. Vue polls the status and downloads the file when the worker is done.

The three jobs



Export as CSV

Button click. User: own attempts. Admin: one row per user with average %.



Daily reminder

Every day 6 PM. Users who did not visit, or new quizzes in the last 24 h.



Monthly report

1st of the month. HTML email: attempts, average, rank, table.

Connect Celery to Flask

backend/application/celery_app.py

```
def celery_init_app(app):
    class FlaskTask(Task):
        def __call__(self, *args, **kwargs):
            with app.app_context():   # db + templates work
                return self.run(*args, **kwargs)

    celery = Celery(app.name, task_cls=FlaskTask)
    celery.config_from_object(app.config["CELERY"])
    celery.conf.beat_schedule = {
        "daily-reminder": {
            "task": "application.tasks.daily_reminders",
            "schedule": crontab(hour=18, minute=0)},
        "monthly-report": {
            "task": "application.tasks.monthly_reports",
            "schedule": crontab(day_of_month=1, hour=8)},
    }
    app.extensions["celery"] = celery
```



FlaskTask

every task runs inside the app context



Redis

broker = queue, backend = results



crontab

beat puts tasks in the queue on time

A task and its API

backend/application/tasks.py · api/jobs.py

```
@shared_task
def export_csv(user_id, is_admin):
    ...                              # build rows from the DB
    with open(os.path.join(EXPORT_DIR, filename), "w") as f:
        csv.writer(f).writerows([header, *rows])
    return filename

@jobs_bp.post("/export")
@auth_required("token")
def start_export():
    task = export_csv.delay(current_user.id,
                            current_user.has_role("admin"))
    return jsonify(task_id=task.id), 202

# GET /export/<task_id>       -> {"state": "SUCCESS"}
# GET /export/<task_id>/file  -> the CSV (owner only)
```



.delay()

puts the task in Redis and returns at once



202 Accepted

job started, not finished



Owner only

users download only their own files

Export flow in the browser



ExportButton.vue is one component used on both the user "My scores" page and the admin "Users" page.

Run it: five terminals

```
Terminal

# 1  queue + results
redis-server
# 2  optional: http://localhost:8025
mailhog
# 3  backend (venv active)
python app.py
# 4  worker
celery -A worker.celery worker -l info
# 5  scheduler
celery -A worker.celery beat -l info
# frontend
npm run dev
```

Check that...

- Export as CSV downloads a file (user and admin)
- The task appears in the worker terminal
- "Send reminders now" → mail in MailHog
- "Send monthly report now" → HTML report
- Windows: run Redis and Celery inside WSL

Common errors

Error	Fix
Export stays "Preparing…" then fails	worker not running, or started before the code change – restart it
Connection refused 6379	Redis not running (redis-cli ping)
Received unregistered task	start the worker with -A worker.celery from the backend folder
Scheduled task never runs	beat not started, or wrong timezone in config

Recap

- Slow work goes to a Celery worker; Flask answers immediately
- Redis is both the queue and the result store
- Celery beat triggers the daily reminder and the monthly report
- Vue polls the task and downloads the CSV when it is ready

Next: Day 7 – Redis caching, summary charts and the final review.